

使用 Jetpack Compose 建構自動調整式應用程式

來源：<https://codelabs.developers.google.com/jetpack-compose-adaptability?hl=zh-tw>

步驟數：6

在本程式碼研究室中，您將瞭解如何建構適用於手機、平板電腦和摺疊式裝置的自動調整應用程式，並探討可連性。您也會學到 Material 3 自動調整元件的最佳做法。

目錄

1. [簡介](#)
2. [做好準備](#)
3. [讓應用程式可適應不同裝置](#)
4. [動態導覽](#)
5. [螢幕空間使用情形](#)
6. [恭喜](#)

步驟 1 · 約 3 分鐘

1. 簡介

在本程式碼研究室中，您將瞭解如何建構自動調整版面配置的應用程式，能夠適用於手機、平板電腦和摺疊式裝置，以及如何使用 Jetpack Compose 提升可連性。另外，您也會學到 Material 3 元件和主題設定的最佳做法。

深入探討前，請務必先瞭解適應性的意義。

適應性

應用程式的 UI 應配合不同的視窗大小、螢幕方向和板型規格做出回應。自動調整式版面配置會配合可用的畫面空間而變化。無論是為填滿空間而稍微調整版面配置、選擇相應的導覽樣式，還是為運用額外空間而徹底變更版面配置，都屬於這類變化。

詳情請參閱[自適應設計](#)。

在本程式碼研究室中，您將探索如何使用 Jetpack Compose，並思考使用 Jetpack Compose 時的適應性。您會建構名為「Reply」的應用程式，瞭解如何針對各種螢幕實作自動調整功能，以及自動調整功能和可及性如何共同運作，為使用者提供最佳體驗。

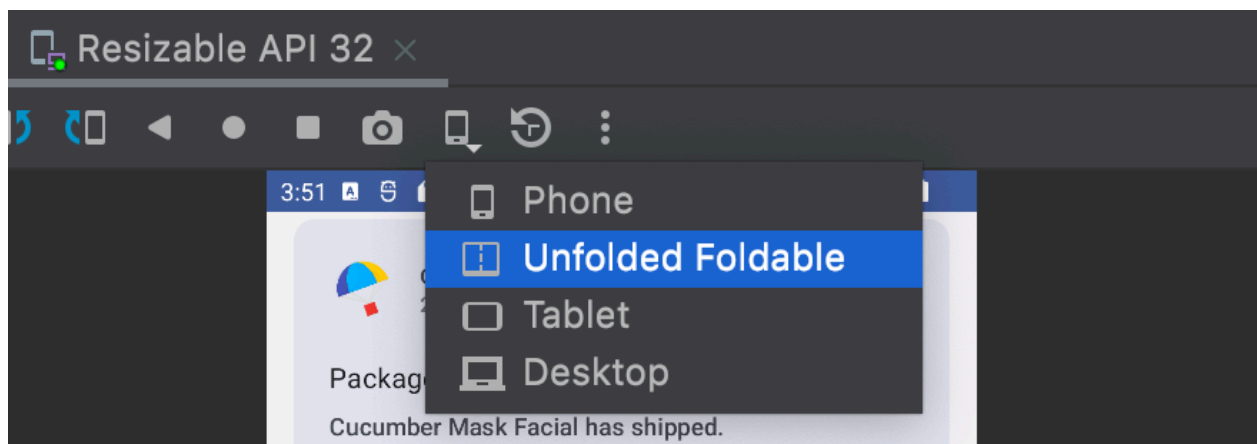
課程內容

- 如何使用 Jetpack Compose 設計應用程式，以支援所有視窗大小。
- 如何為不同折疊式裝置指定應用程式。
- 如何使用不同類型的導覽，提升可及性和無障礙功能。
- 如何使用 Material 3 元件，為各種視窗大小提供最佳體驗。

軟硬體需求

- 最新的 [Android Studio](#) 穩定版。
- [Android 13 可調整大小的虛擬裝置](#)。
- 具備 Kotlin 知識。
- 瞭解 Compose 的基本知識 (例如 `@Composable` 註解)。
- 熟悉 Compose 版面配置的基本知識 (例如 `Row` 和 `Column`)。
- 熟悉修飾符的基本概念 (例如 `Modifier.padding()`)。

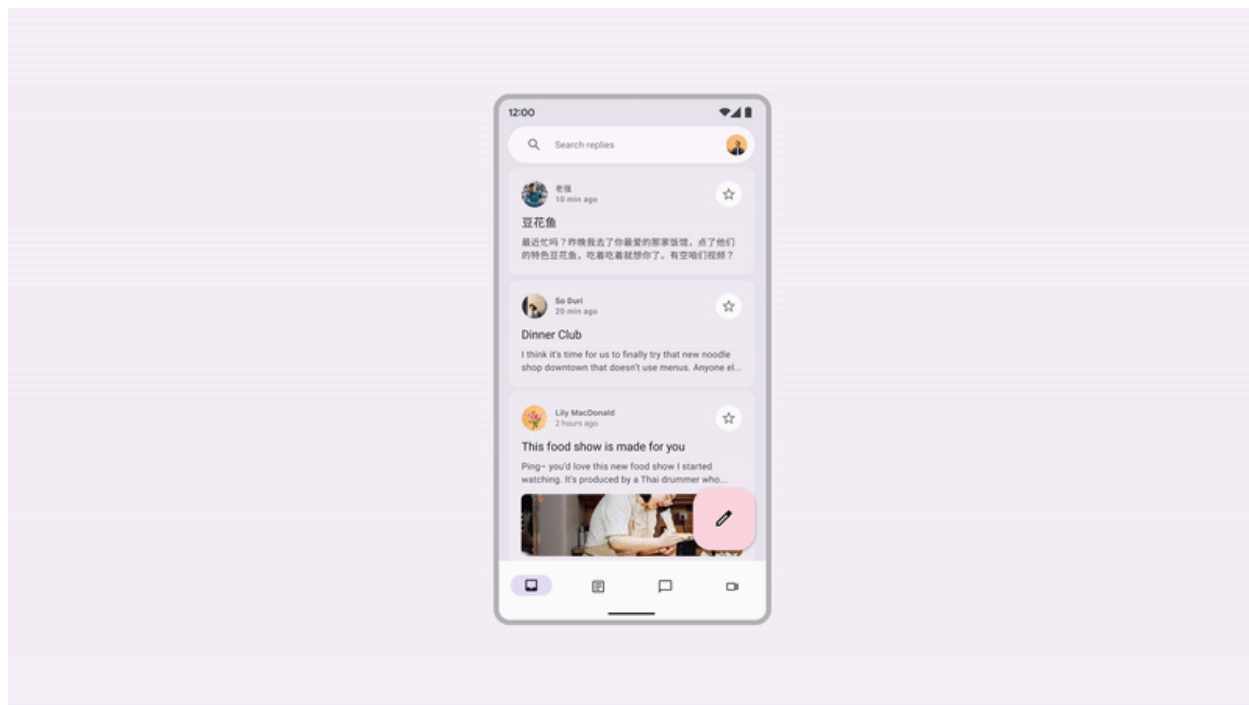
在本程式碼研究室中，您將使用[可調整大小的模擬器](#)，在不同類型的裝置和視窗大小之間切換。



如果您不熟悉 Compose，建議先完成 [Jetpack Compose 基本概念程式碼研究室](#)，再進行本程式碼研究室。

建構項目

- 這個互動式電子郵件用戶端應用程式名為 Reply，採用自動調整式設計的最佳做法、不同的 Material Design 導覽方式，以及最佳螢幕空間使用方式。



步驟 2 · 約 5 分鐘

2. 做好準備

如要取得本程式碼實驗室的程式碼，請從指令列複製 GitHub 存放區：

```
git clone https://github.com/android/codelab-android-compose.git
cd codelab-android-compose/AdaptiveUiCodelab
```

或者，您也可以將存放區下載為 ZIP 檔案：

`codelab-android-compose` 存放區包含該路徑中所有程式碼研究室的範例程式碼。

在本程式碼研究室中，請使用 `AdaptiveUiCodelab` 專案。專案在多個 Git 分支版本中建構而成：

- **main**：包含本專案的範例程式碼。您必須修改程式碼，才能完成程式碼研究室。
- **end** - 內含本程式碼研究室的解決方案。

建議您先從 **main** 分支版本的程式碼著手，依自己的步調逐步完成本程式碼研究室。

在 Android Studio 中開啟專案

1. 在「Welcome to Android Studio」視窗中，選取「



Open an Existing Project」。

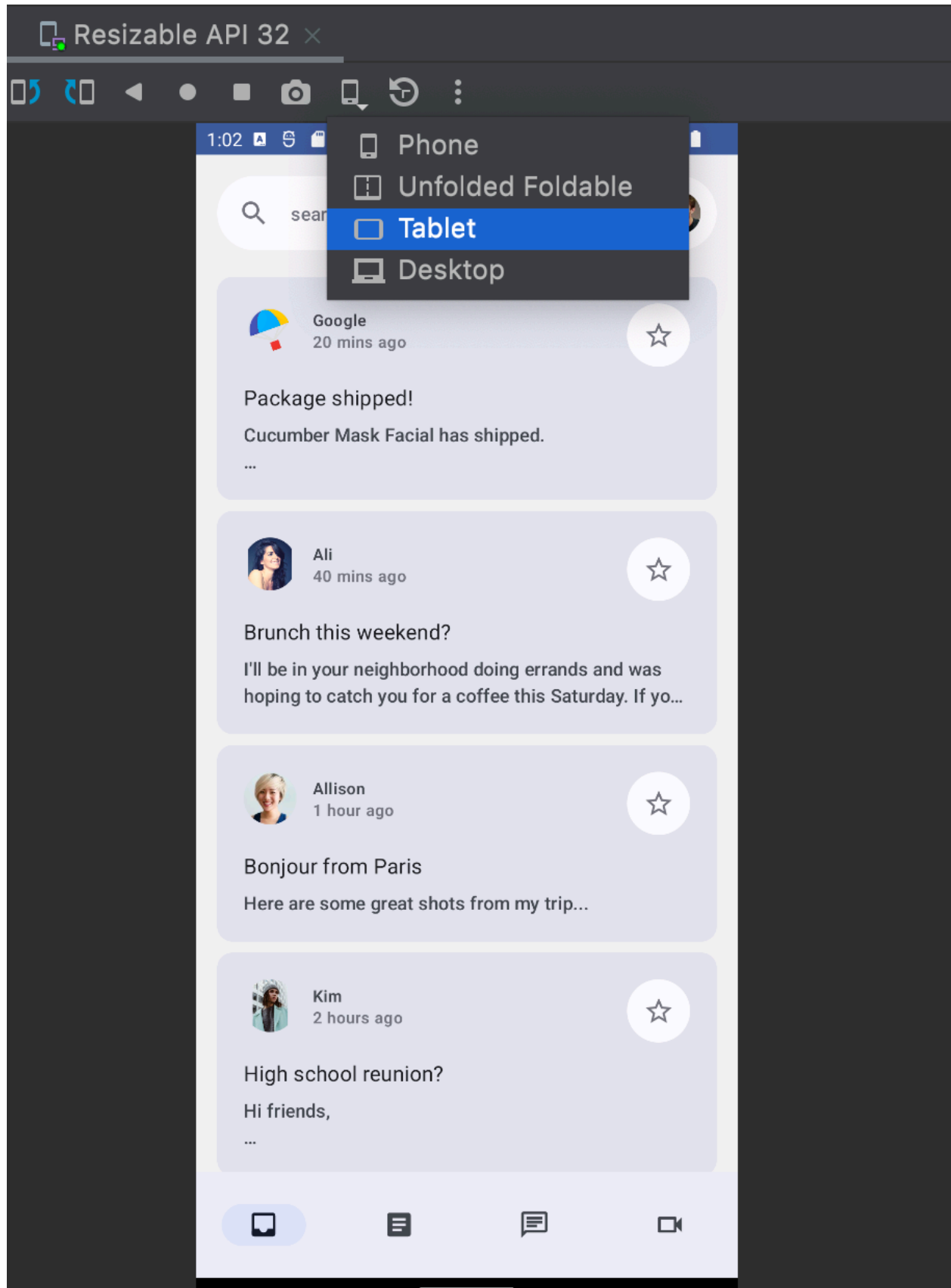
2. 選取資料夾 `<Download Location>/AdaptiveUiCodelab` (請務必選取內含 `build.gradle` 的 `AdaptiveUiCodelab` 目錄)。
3. Android Studio 匯入專案後，請測試是否能執行 **main** 分支版本。

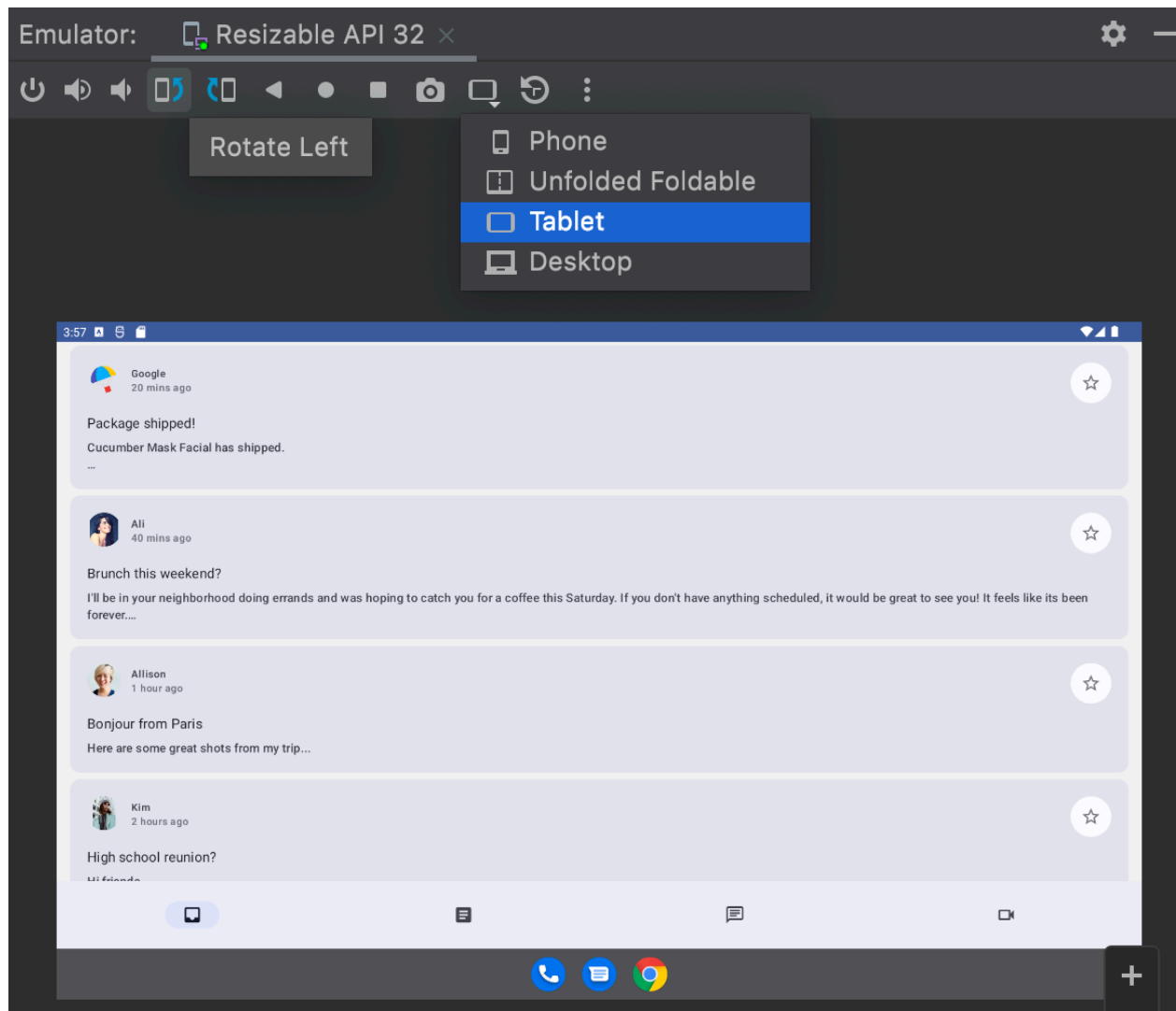
探索範例程式碼

main 分支版本程式碼包含 **ui** 套件。您將使用該套件中的下列檔案：

- **MainActivity.kt**：用於啟動應用程式的進入點活動。
- **ReplyApp.kt**：包含主畫面 UI 可組合函式。
- **ReplyHomeViewModel.kt** - 提供應用程式內容的資料和 UI 狀態。
- **ReplyListContent.kt** - 包含提供清單和詳細資料畫面的可組合函式。

如果您在可調整大小的模擬器上執行這個應用程式，並嘗試使用不同類型的裝置 (例如手機或平板電腦)，UI 只會擴展至指定空間，而不會充分利用螢幕空間或提供可及性人體工學。





您將更新這個畫面，充分利用螢幕空間、提升可用性，並改善整體使用者體驗。

3. 讓應用程式可適應不同裝置

本節將介紹應用程式的適應性，以及 Material 3 提供的元件，讓您更輕鬆地達成這個目標。此外，本單元也會介紹您要鎖定的螢幕類型和狀態，包括手機、平板電腦、大型平板電腦和折疊式裝置。

首先，您會瞭解視窗大小、摺疊姿勢和不同類型的導覽選項。接著，您就能在應用程式中使用這些 API，讓應用程式更具適應性。

視窗大小

Android 裝置的外形與大小不盡相同，包括手機、摺疊式裝置、平板電腦和 ChromeOS 裝置。如要盡可能支援多種視窗大小，UI 需要採用回應式及自動調整式設計。為協助您找出適當的閾值：門檻來變更應用程式的 UI，我們定義了中斷點值，可將裝置分類為預先定義的大小類別 (精簡、中等和展開)，稱為[視窗大小類別](#)。這是一組自主的可視區域中斷點，有助於設計、開發及測試回應式與自動調整式應用程式版面配置。

這些類別經過特別挑選，可讓版面配置保持簡單又兼具彈性，進一步根據獨特的情境需求來最佳化應用程式。視窗大小類別完全取決於應用程式可用的螢幕空間，這不一定是進行多工處理或顯示其他分隔內容時的整個實體螢幕。



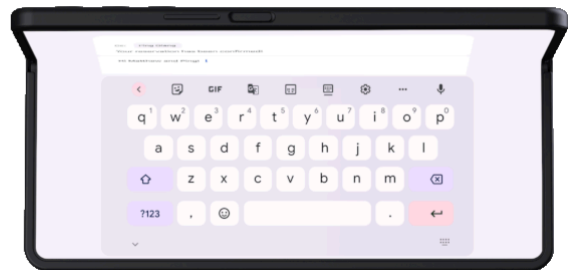
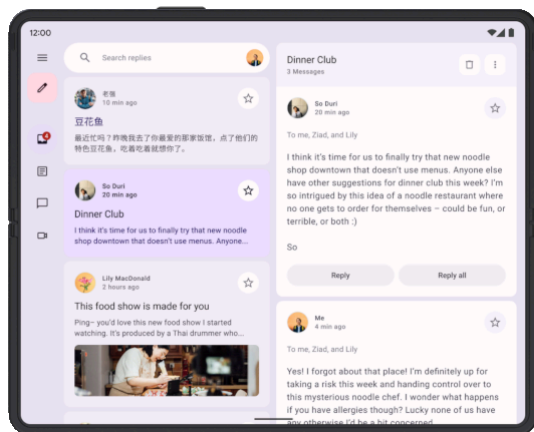
Height Window Size Classes



寬度和高度會另外分類，因此應用程式隨時都有兩種視窗大小類別：寬度類別和高度類別。一般來說，可用寬度比可用高度更重要，這是因為直向捲動操作比較常見，因此在這個案例中，您也會使用寬度大小類別。

摺疊狀態

摺疊式裝置的尺寸和鉸鏈位置各不相同，因此應用程式可適應的情況更多。轉軸可能會遮蔽部分螢幕，導致該區域不適合顯示內容；轉軸也可能分隔螢幕，也就是說，裝置展開時會有兩個獨立的實體螢幕。



此外，使用者也可能在螢幕部分開啟時查看內螢幕，因此會根據摺疊方向採取不同的姿勢：桌上型姿勢 (水平摺疊，如上圖右側所示) 和書本姿勢 (垂直摺疊)。

進一步瞭解[摺疊型態和鉸鏈](#)。

導入支援摺疊裝置的自動調整式版面配置時，請務必考量上述事項。

取得自動調整資訊

Material3 [adaptive](#) 程式庫可讓您輕鬆存取應用程式執行所在視窗的相關資訊。

1. 將這個構件及其版本新增至版本目錄檔案：

`gradle/libs.versions.toml`

```
[versions]
material3Adaptive = "1.0.0"

[libraries]
androidx-material3-adaptive = { module = "androidx.compose.material3.adaptive:adaptive",
version.ref = "material3Adaptive" }
```

2. 在應用程式模組的建構檔案中，新增程式庫依附元件，然後執行 Gradle 同步作業：

app/build.gradle.kts

```
dependencies {

    implementation(libs.androidx.material3.adaptive)
}
```

現在，您可以在任何可組合項範圍中使用 `currentWindowAdaptiveInfo()`，取得包含目前視窗大小類別等資訊的 `WindowAdaptiveInfo` 物件，以及裝置是否處於摺疊型態 (例如桌面型態)。

你現在可以在 `MainActivity` 試用這項功能。

3. 在 `ReplyTheme` 區塊內的 `onCreate()` 中，取得視窗適應性資訊，並在 `Text` 可組合函式中顯示大小類別。您可以在 `ReplyApp()` 元素後新增這項內容：

MainActivity.kt

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    setContent {
        ReplyTheme {
            val uiState by viewModel.uiState.collectAsStateWithLifecycle()
            ReplyApp(
                replyHomeUIState = uiState,
                onEmailClick = viewModel::setSelectedEmail
            )

            val adaptiveInfo = currentWindowAdaptiveInfo()
            val sizeClassText =
                "${adaptiveInfo.windowSizeClass.windowWidthSizeClass}\n" +
                "${adaptiveInfo.windowSizeClass.windowHeightSizeClass}"
            Text(
                text = sizeClassText,
                color = Color.Magenta,
                modifier = Modifier.padding(
                    WindowInsets.safeDrawing.asPaddingValues()
                )
            )
        }
    }
}
```

現在執行應用程式，應用程式內容上就會顯示列印的視窗大小類別。歡迎探索視窗調適資訊提供的其他內容。之後您可以移除這個 `Text`，因為它會涵蓋應用程式內容，而且後續步驟不需要用到。

4. 動態導覽

現在，您要配合裝置狀態和大小變化調整應用程式的導覽，讓應用程式更容易使用。

使用者拿著手機時，手指通常會位於螢幕底部。使用者拿著開啟的摺疊式裝置或平板電腦時，手指通常會靠近兩側。使用者應能瀏覽應用程式或與應用程式互動，而不必將手擺放在極端的位置或改變手部擺放方式。

設計應用程式並決定要在版面配置中放置互動式 UI 元素時，請考慮螢幕不同區域的人體工學影響。

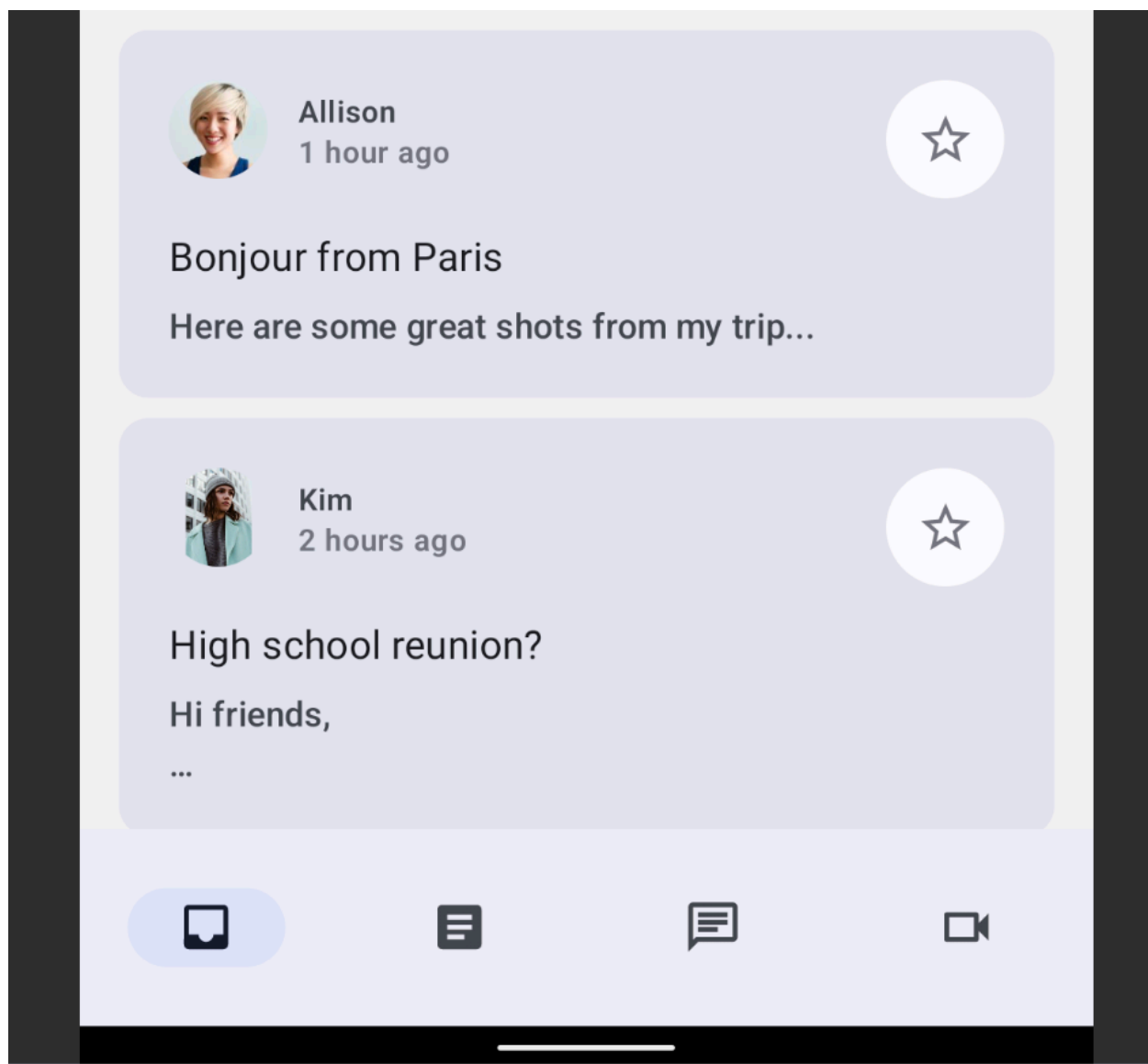
- 拿著裝置時，哪些區域最容易觸及？
- 哪些區域只能透過伸長手指觸及，可能不太方便？
- 哪些區域難以觸及，或距離使用者握持裝置的位置很遠？

導覽是使用者最先互動的項目，其中包含與重要使用者歷程相關的高重要性動作，因此應放置在最容易觸及的區域。

Material 自適應程式庫提供多種元件，可協助您根據裝置的視窗大小類別實作導覽功能。

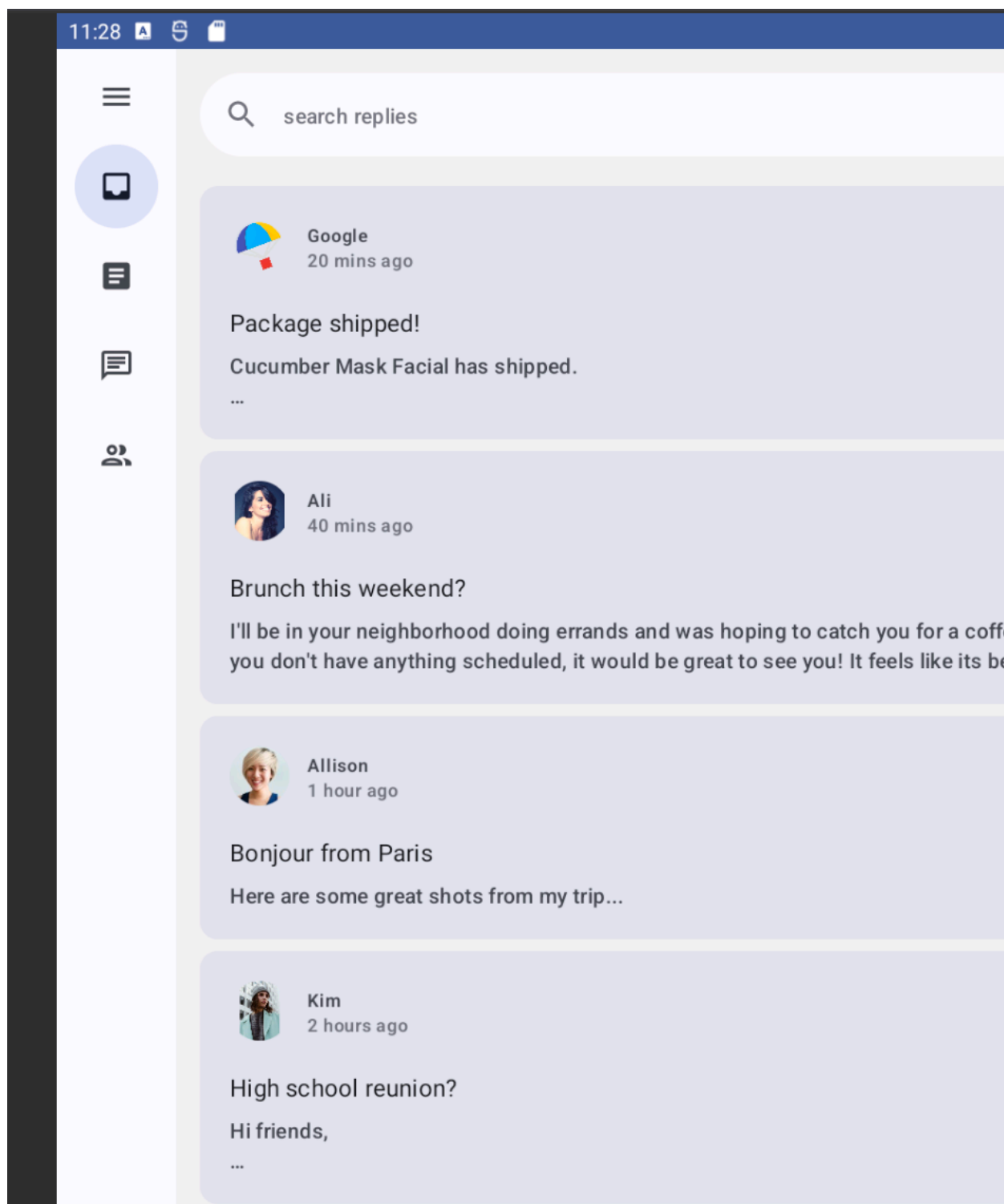
底部導覽

底部導覽非常適合精簡尺寸，因為我們自然會握持裝置，讓大拇指輕鬆觸及所有底部導覽觸控點。只要裝置尺寸較小，或是摺疊式裝置處於摺疊狀態，即可使用這項功能。



導覽邊欄

對於中等寬度的視窗大小，導覽軌非常適合觸及，因為拇指自然會落在裝置側邊。您也可以將導覽軌與導覽匣合併，顯示更多資訊。

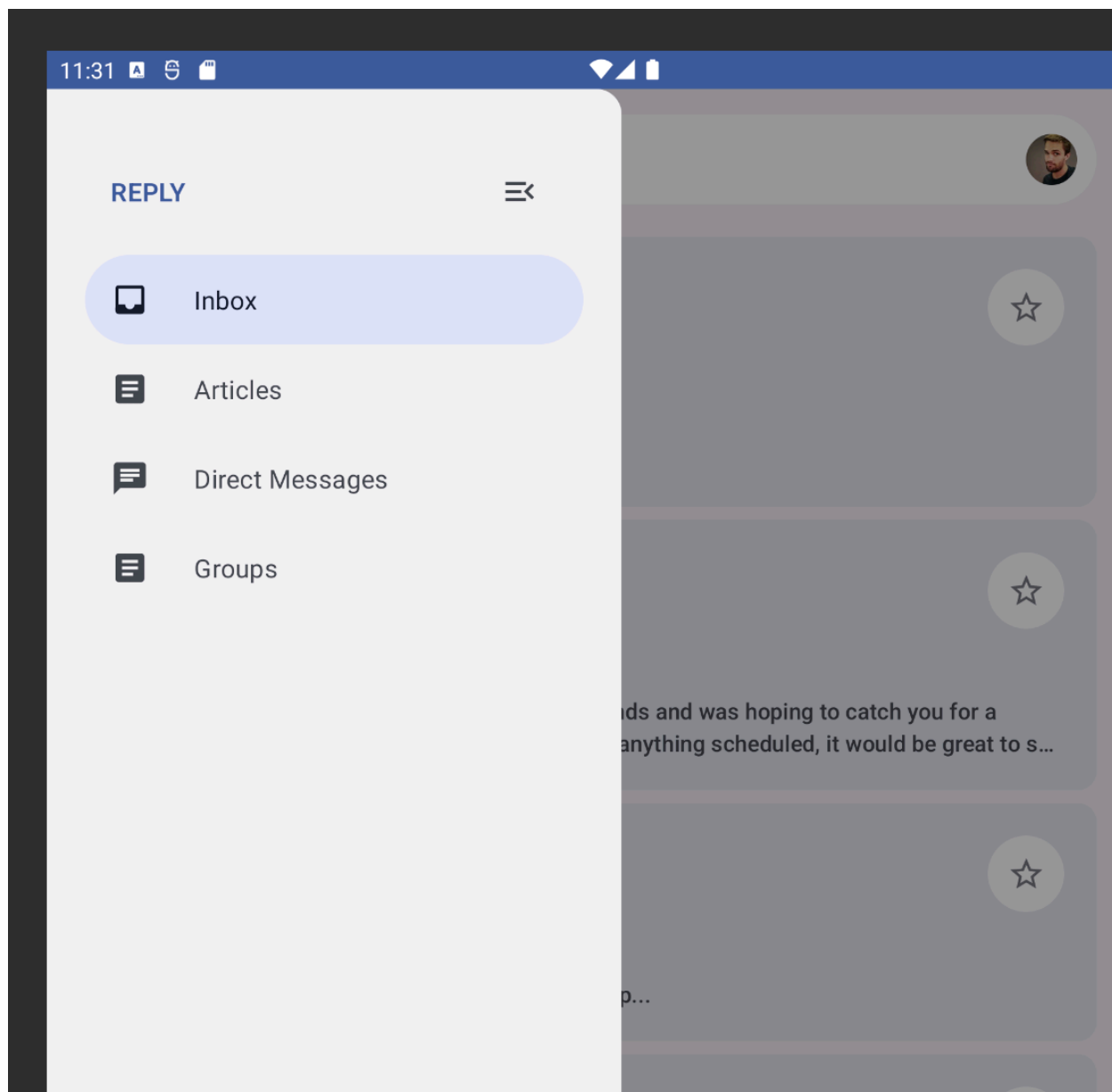


導覽匣

使用平板電腦或大型裝置時，導覽匣可讓您輕鬆查看導覽分頁的詳細資訊，導覽匣有兩種：強制回應導覽匣和永久導覽匣。

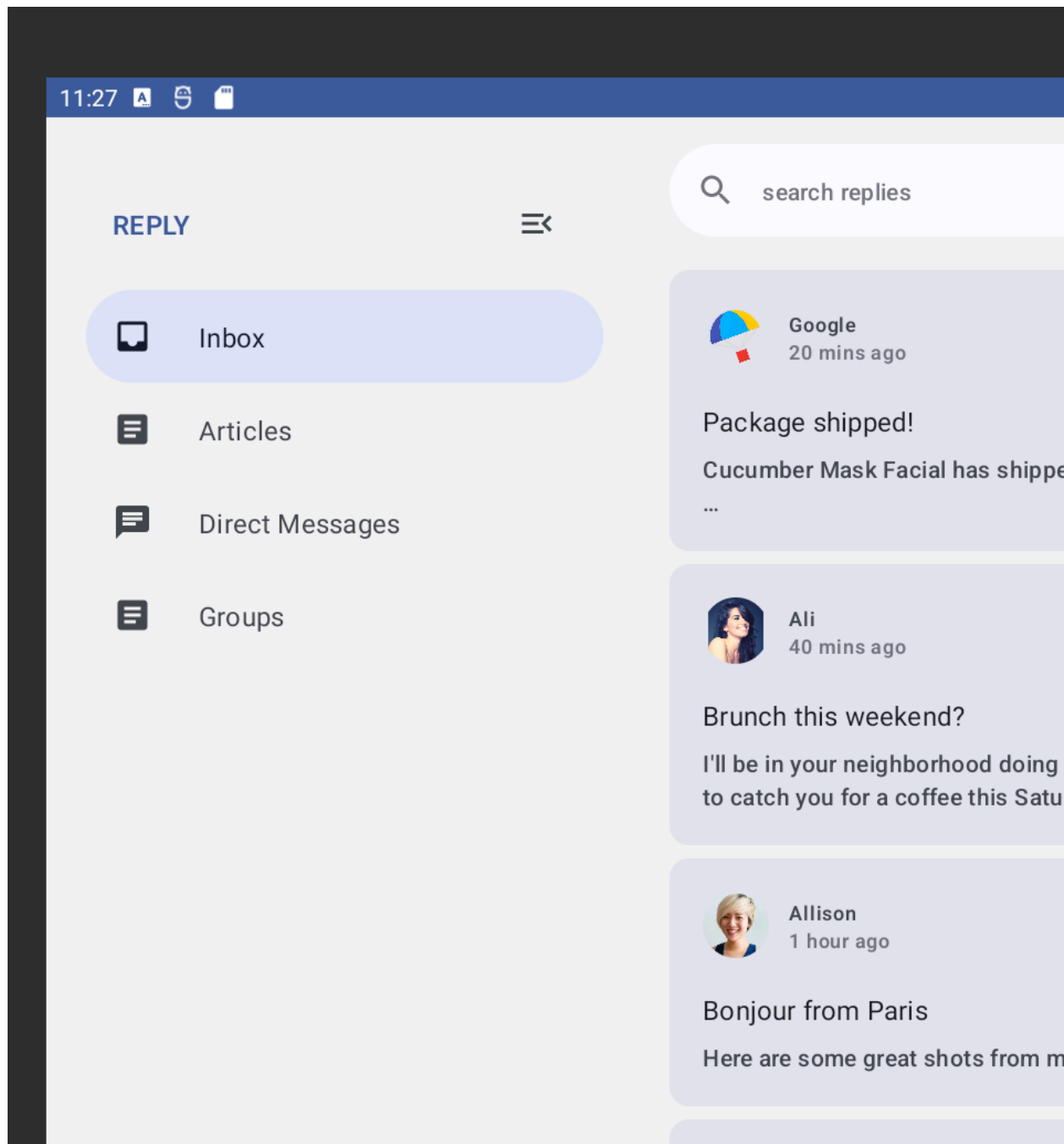
強制回應導覽匣

對於螢幕尺寸較小到中等的手機和平板電腦，您可以使用模式導覽匣，因為這類裝置的內容可以展開或隱藏為疊加層。有時可以與導覽軌搭配使用。



固定式導覽匣

您可以在大型平板電腦、Chromebook 和桌機上使用永久導覽匣，固定顯示導覽功能。



實作動態導覽

現在，您將隨著裝置狀態和大小的變化，在不同類型的導覽之間切換。

目前，無論裝置狀態為何，應用程式一律會在螢幕內容下方顯示 `NavigationBar`。您可以改用 `Material NavigationSuiteScaffold` 元件，根據目前視窗大小類別等資訊，自動切換不同的導覽元件。

1. 更新版本目錄和應用程式的建構指令碼，然後執行 Gradle 同步作業，即可新增 Gradle 依附元件來取得這個元件：

gradle/libs.versions.toml

```
[versions]
material3AdaptiveNavSuite = "1.3.0"

[libraries]
androidx-material3-adaptive-navigation-suite = { module =
"androidx.compose.material3:material3-adaptive-navigation-suite", version.ref =
"material3AdaptiveNavSuite" }
```

app/build.gradle.kts

```
dependencies {

    implementation(libs.androidx.material3.adaptive.navigation.suite)
}
```

2. 在 **ReplyApp.kt** 中找出 `ReplyNavigationWrapper()` 可組合函式，並將 `Column` 及其內容替換為 `NavigationSuiteScaffold`：

ReplyApp.kt


```

@Composable
private fun ReplyNavigationWrapperUI(
    content: @Composable () -> Unit = {}
) {
    var selectedDestination: ReplyDestination by remember {
        mutableStateOf(ReplyDestination.Inbox)
    }

    NavigationSuiteScaffold(
        navigationSuiteItems = {
            ReplyDestination.entries.forEach {
                item(
                    selected = it == selectedDestination,
                    onClick = { /*TODO update selection*/ },
                    icon = {
                        Icon(
                            imageVector = it.icon,
                            contentDescription = stringResource(it.labelRes)
                        )
                    },
                    label = {
                        Text(text = stringResource(it.labelRes))
                    },
                )
            }
        }
    ) {
        content()
    }
}

```

`navigationSuiteItems` 引數是區塊，可讓您使用 `item()` 函式新增項目，與在 `LazyColumn` 中新增項目類似。在結尾的 `lambda` 中，這段程式碼會呼叫傳遞為 `ReplyNavigationWrapperUI()` 引數的 `content()`。

在模擬器上執行應用程式，並嘗試在手機、摺疊式裝置和平板電腦之間變更大小，您會看到導覽列變更為導覽軌，然後再變回導覽列。

在非常寬的視窗 (例如橫向平板電腦) 上，您可能想顯示永久導覽匣。`NavigationSuiteScaffold` 支援顯示永久導覽匣，但目前 `WindowWidthSizeClass` 值均未顯示。不過，只要稍微修改，就能讓 Google 助理執行這項操作。

3. 在呼叫 `NavigationSuiteScaffold` 之前加入下列程式碼：

ReplyApp.kt

```

@Composable
private fun ReplyNavigationWrapperUI(
    content: @Composable () -> Unit = {}
) {
    var selectedDestination: ReplyDestination by remember {
        mutableStateOf(ReplyDestination.Inbox)
    }

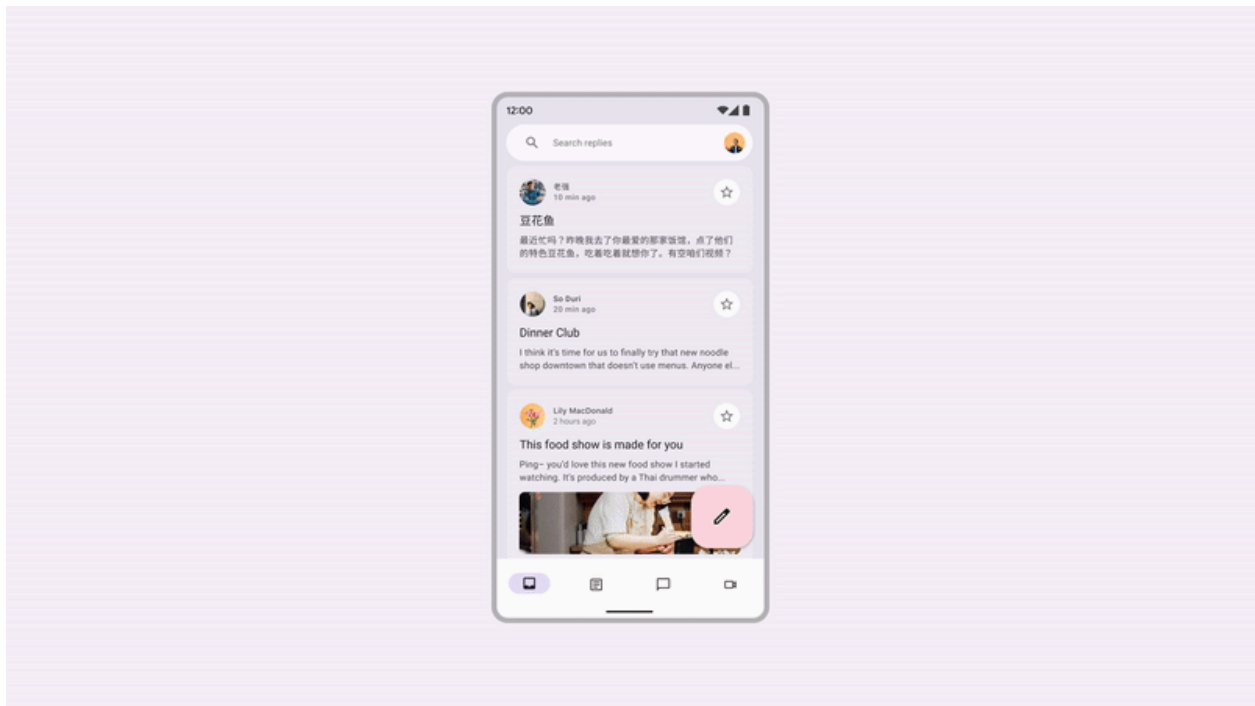
    val windowSize = with(LocalDensity.current) {
        currentWindowSize().toSize().toDpSize()
    }
    val layoutType = if (windowSize.width >= 1200.dp) {
        NavigationSuiteType.NavigationDrawer
    } else {
        NavigationSuiteScaffoldDefaults.calculateFromAdaptiveInfo(
            currentWindowAdaptiveInfo()
        )
    }

    NavigationSuiteScaffold(
        layoutType = layoutType,
        ...
    ) {
        content()
    }
}

```

這段程式碼會先取得視窗大小，然後使用 `currentWindowSize()` 和 `LocalDensity.current` 將視窗大小轉換為 DP 單位，接著比較視窗寬度，決定導覽 UI 的版面配置類型。如果視窗寬度至少為 `1200.dp`，則會使用 `NavigationSuiteType.NavigationDrawer`。否則會改用預設計算方式。

在可調整大小的模擬器上再次執行應用程式，並試用不同類型，您會發現每當螢幕設定變更或展開摺疊式裝置時，導覽會變更為適合該大小的類型。



恭喜！您已瞭解如何使用不同類型的導覽功能，支援不同類型的視窗大小和狀態！

在下一節中，您將瞭解如何充分利用剩餘的螢幕區域，而不是將相同的清單項目從一端延伸到另一端。

5. 螢幕空間使用情形

無論是在小型平板電腦、展開的裝置還是大型平板電腦上執行應用程式，螢幕都會延展以填滿剩餘空間。您希望充分利用螢幕空間顯示更多資訊，例如在這個應用程式中，在同一頁面上向使用者顯示電子郵件和討論串。

Material 3 定義了三種[標準版面配置](#)，每種版面配置都適用於精簡、中等和展開的視窗大小類別。[清單/詳細資料](#)標準版面配置非常適合這個用途，且可做為 `ListDetailPaneScaffold` 使用。

1. 如要取得這個元件，請新增下列依附元件並執行 Gradle 同步作業：

gradle/libs.versions.toml

```
[libraries]
androidx-material3-adaptive-layout = { module =
    "androidx.compose.material3.adaptive:adaptive-layout", version.ref = "material3Adaptive" }
androidx-material3-adaptive-navigation = { module =
    "androidx.compose.material3.adaptive:adaptive-navigation", version.ref = "material3Adaptive"
}
```

app/build.gradle.kts

```
dependencies {

    implementation(libs.androidx.material3.adaptive.layout)
    implementation(libs.androidx.material3.adaptive.navigation)
}
```

2. 在 `ReplyApp.kt` 中找出 `ReplyAppContent()` 可組合函式，目前該函式只會呼叫 `ReplyListPane()` 來顯示清單窗格。插入下列程式碼，將這個實作項目替換為 `ListDetailPaneScaffold`。由於這是實驗性 API，您也將在 `ReplyAppContent()` 函式中加入 `@OptIn` 註解：

ReplyApp.kt

```

@OptIn(ExperimentalMaterial3AdaptiveApi::class)
@Composable
fun ReplyAppContent(
    replyHomeUIState: ReplyHomeUIState,
    onEmailClick: (Email) -> Unit,
) {
    val navigator = rememberListDetailPaneScaffoldNavigator<Long>()

    ListDetailPaneScaffold(
        directive = navigator.scaffoldDirective,
        value = navigator.scaffoldValue,
        listPane = {
            ReplyListPane(replyHomeUIState, onEmailClick)
        },
        detailPane = {
            ReplyDetailPane(replyHomeUIState.emails.first())
        }
    )
}

```

這段程式碼會先使用 `rememberListDetailPaneNavigator()` 建立導覽器。導覽器可控制要顯示哪個窗格，以及該窗格應呈現的內容，稍後會進行示範。

當視窗寬度大小類別為展開時，`ListDetailPaneScaffold` 會顯示兩個窗格。否則，系統會根據為兩個參數 (即支架指令和支架值) 提供的值，顯示其中一個窗格。如要取得預設行為，這段程式碼會使用架構指令和導覽器提供的架構值。

其餘必要參數是窗格的可組合 Lambda。`ReplyListPane()` 和 `ReplyDetailPane()` (位於 `ReplyListContent.kt` 中) 分別用於填入清單和詳細資料窗格的角色。`ReplyDetailPane()` 預期會收到電子郵件引數，因此目前這段程式碼會使用 `ReplyHomeUIState` 中電子郵件清單的第一封電子郵件。

執行應用程式，並將模擬器檢視畫面切換為摺疊式裝置或平板電腦 (您可能也必須變更螢幕方向)，即可查看雙窗格版面配置。這樣看起來比之前好多了！

現在，讓我們處理這個畫面的一些所需行為。使用者輕觸清單窗格中的電子郵件時，詳細資料窗格應顯示該郵件和所有回覆。目前應用程式不會追蹤選取的電子郵件，輕觸項目也不會執行任何動作。建議將這項資訊與其餘 UI 狀態一起存放在 `ReplyHomeUIState` 中。

3. 開啟 `ReplyHomeViewModel.kt` 並找出 `ReplyHomeUIState` 資料類別。為所選電子郵件新增屬性，預設值為 `null`：

ReplyHomeViewModel.kt

```

data class ReplyHomeUIState(
    val emails : List<Email> = emptyList(),
    val selectedEmail: Email? = null,
    val loading: Boolean = false,
    val error: String? = null
)

```

4. 在同一個檔案中，`ReplyHomeViewModel` 具有 `setSelectedEmail()` 函式，使用者輕觸清單項目時會呼叫該函式。修改這個函式，複製 UI 狀態並記錄所選電子郵件：

ReplyHomeViewModel.kt

```
fun setSelectedEmail(email: Email) {
    _uiState.update {
        it.copy(selectedEmail = email)
    }
}
```

請注意，使用者輕觸任何項目之前，以及所選電子郵件為 `null` 時，會發生什麼情況。詳細資料窗格中應顯示哪些內容？處理這個情況的方法有很多種，例如預設顯示清單中的第一個項目。

5. 在同一個檔案中，修改 `observeEmails()` 函式。載入電子郵件清單時，如果先前的 UI 狀態沒有選取的電子郵件，請將其設為第一個項目：

ReplyHomeViewModel.kt

```
private fun observeEmails() {
    viewModelScope.launch {
        emailsRepository.getAllEmails()
            .catch { ex ->
                _uiState.value = ReplyHomeUIState(error = ex.message)
            }
            .collect { emails ->
                val currentSelection = _uiState.value.selectedEmail
                _uiState.value = ReplyHomeUIState(
                    emails = emails,
                    selectedEmail = currentSelection ?: emails.first()
                )
            }
    }
}
```

6. 返回 `ReplyApp.kt`，並使用選取的電子郵件 (如有)，填入詳細資料窗格內容：

ReplyApp.kt

```
ListDetailPaneScaffold(
    // ...
    detailPane = {
        if (replyHomeUIState.selectedEmail != null) {
            ReplyDetailPane(replyHomeUIState.selectedEmail)
        }
    }
)
```

再次執行應用程式，將模擬器切換為平板電腦大小，然後輕觸清單項目，查看詳細資料窗格的內容是否更新。

如果兩個窗格都顯示在畫面上，這項功能運作良好，但如果視窗只能顯示一個窗格，輕觸項目時，畫面看起來不會有任何變化。請嘗試將模擬器檢視畫面切換為手機或直向摺疊式裝置，並注意即使輕觸項目，也只會顯示清單窗格。這是因為即使所選電子郵件已更新，`ListDetailPaneScaffold` 仍會將焦點放在這些設定的清單窗格。

7. 如要修正這個問題，請將下列程式碼插入傳遞至 `ReplyListPane` 的 lambda 中：

ReplyApp.kt

```

ListDetailPaneScaffold(
    // ...
    listPane = {
        ReplyListPane(
            replyHomeUIState = replyHomeUIState,
            onEmailClick = { email ->
                onEmailClick(email)
                navigator.navigateTo(ListDetailPaneScaffoldRole.Detail, email.id)
            }
        )
    },
    // ...
)

```

這個 lambda 會使用先前建立的導覽器，在點選項目時新增其他行為。這個函式會呼叫傳遞至此函式的原始 lambda，然後呼叫 `navigator.navigateTo()`，指定要顯示哪個窗格。支架中的每個窗格都有相關聯的角色，詳細資料窗格的角色則是 `ListDetailPaneScaffoldRole.Detail`。在較小的視窗中，這會讓應用程式看起來像是已向前導覽。

應用程式也必須處理使用者從詳細資料窗格按下返回按鈕時的情況，而這項行為會因顯示一個或兩個窗格而有所不同。

8. 新增下列程式碼，支援返回瀏覽。

ReplyApp.kt

```

@OptIn(ExperimentalMaterial3AdaptiveApi::class)
@Composable
fun ReplyAppContent(
    replyHomeUIState: ReplyHomeUIState,
    onEmailClick: (Email) -> Unit,
) {
    val navigator = rememberListDetailPaneScaffoldNavigator<Long>()

    BackHandler(navigator.canNavigateBack()) {
        navigator.navigateBack()
    }

    ListDetailPaneScaffold(
        directive = navigator.scaffoldDirective,
        value = navigator.scaffoldValue,
        listPane = {
            AnimatedPane {
                ReplyListPane(
                    replyHomeUIState = replyHomeUIState,
                    onEmailClick = { email ->
                        onEmailClick(email)
                        navigator.navigateTo(ListDetailPaneScaffoldRole.Detail, email.id)
                    }
                )
            }
        },
        detailPane = {
            AnimatedPane {
                if (replyHomeUIState.selectedEmail != null) {
                    ReplyDetailPane(replyHomeUIState.selectedEmail)
                }
            }
        }
    )
}

```

導覽器會知道 `ListDetailPaneScaffold` 的完整狀態、是否可返回瀏覽，以及在所有這些情況下該怎麼做。這段程式碼會建立 `BackHandler`，只要導覽器可以返回，就會啟用該項目，並在 `lambda` 內呼叫 `navigateBack()`。此外，為了讓窗格之間的轉場效果更加流暢，每個窗格都會包裝在 `AnimatedPane()` 可組合函式中。

在可調整大小的模擬器上，針對所有不同類型的裝置再次執行應用程式，並注意每當螢幕設定變更或展開摺疊式裝置時，導覽和螢幕內容會動態變更，以回應裝置狀態變更。此外，請試著輕觸清單窗格中的電子郵件，看看版面配置在不同畫面上的行為，例如並排顯示兩個窗格，或在兩者之間順暢地切換。



恭喜！您已成功讓應用程式適應各種裝置狀態和大小。請盡情試用，在摺疊式裝置、平板電腦或其他行動裝置上執行應用程式。

6. 恭喜

恭喜！您已成功完成本程式碼研究室，並瞭解如何使用 Jetpack Compose 打造自動調整式應用程式。

您已瞭解如何檢查裝置大小和摺疊狀態，並據此更新應用程式的 UI、導覽和其他功能。您也瞭解了適應性如何提升觸及率及改善使用者體驗。

後續步驟

請參閱 [Compose 課程](#) 中的其他程式碼研究室。

範例應用程式

- [Compose 範例](#) 包含許多應用程式，這些應用程式都採用程式碼研究室中說明的最佳做法。

參考文件

- [Material 3 自動調整式設計](#)
 - [大螢幕專用設計](#)
 - [隆重推出 Material Design 主題設定建構工具](#)
 - [Material 3：推出 Material You](#)
-